

*Corrigé — Devoir 2nde : variables, print, input**Exercice 1 — Types des variables*

- ▶ `a = 5` → type `int` (entier)
- ▶ `b = 3.5` → type `float` (flottant / décimal)
- ▶ `c = "python"` → type `str` (chaîne de caractères)
- ▶ `d = (5 > 2)` → type `bool` (booléen). La comparaison `5 > 2` est vraie, donc `d = True`.

*Exercice 2 — Traces d'exécution*

Programme 1 :

- ▶ `a = 4`, `b = 3 × 4 - 5 = 7` → affiche : `b = 7`

Programme 2 :

- ▶ `c = "bon" + "soir" = "bonsoir"`
- ▶ `d = "bon" * 4 = "bonbonbonbon"`
- ▶ Affiche successivement : `bonsoir` puis `bonbonbonbon`

Programme 3 :

- ▶ Échange de valeurs avec la variable temporaire `c` :
- ▶ `c = a = 22` ; `a = b = 17` ; `b = c = 22`
- ▶ Affiche : `a = 17` et `b = 22`

*Exercice 3 — Programmes avec input*

1. Le programme qui provoque une erreur est le **programme 2** :
  - ▶ `a = input(...)` renvoie une **chaîne de caractères** (type `str`). L'instruction `b = a + 1` tente d'additionner une chaîne et un entier → `TypeError`.
2. Les programmes 1 et 3 fonctionnent :
  - ▶ **Programme 1** : `int(input(...))` convertit la saisie en entier. Si on saisit 5, `b = 5 + 1 = 6` → affiche : `nombre suivant: 6`
  - ▶ **Programme 3** : `a + '1'` concatène deux chaînes. Si on saisit 5, `b = "5" + "1" = "51"` → affiche : `nombre suivant: 51` (pas 6 — ce n'est pas une addition !)

*Exercice 4 — Le triangle rectangle*

Programme complété :

```
1 from math import *
2 a = float(input("Premier cote de l'angle droit : "))
3 b = float(input("Deuxieme cote de l'angle droit : "))
4 hypotenuse = sqrt(a**2 + b**2)
```

```

5 perimetre = a + b + hypotenuse
6 aire      = (a * b) / 2
7 print("Hypotenuse : ", hypotenuse)
8 print("Perimetre   : ", perimetre)
9 print("Aire        : ", aire)

```

► La fonction `sqrt()` du module `math` calcule la racine carrée. L'hypoténuse vaut  $\sqrt{a^2 + b^2}$  (théorème de Pythagore). L'aire d'un triangle rectangle vaut  $\frac{a \times b}{2}$ .

### Exercice 5 — Conversion secondes $\rightarrow$ h/min/s

Programme complété :

```

1 total_seconds = int(input("Entrez la duree en secondes : "))
2 heures       = total_seconds // 3600
3 reste        = total_seconds % 3600
4 minutes      = reste // 60
5 secondes     = reste % 60
6 print(heures, "heures", minutes, "minutes", secondes, "secondes")

```

► **Logique** : 1 heure = 3600 s. Le quotient entier `total_seconds // 3600` donne les heures. Le reste `total_seconds % 3600` est le résidu en secondes, qu'on divise à nouveau par 60 pour obtenir les minutes et les secondes restantes.

► **Test 5643 s** :  $5643 // 3600 = 1$  h, reste =  $5643 - 3600 = 2043$  s ;  $2043 // 60 = 34$  min, reste =  $2043 - 2040 = 3$  s  $\rightarrow$  1 heures 34 minutes 3 secondes

► **Test 2457 s** :  $2457 // 3600 = 0$  h, reste = 2457 s ;  $2457 // 60 = 40$  min, reste =  $2457 - 2400 = 57$  s  $\rightarrow$  0 heures 40 minutes 57 secondes

### Exercice 6 — Résolution de $ax + b = c$

Programme complété :

```

1 # Demander les nombres a, b et c
2 a = float(input("Entrez a : "))
3 b = float(input("Entrez b : "))
4 c = float(input("Entrez c : "))
5 # a est different de 0
6 assert (a != 0)
7 # solution de l'equation ax + b = c
8 x = (c - b) / a
9 # Affichage de la solution de l'equation
10 print("La solution de l'equation est x =", x)

```

1. a. Pour  $a = 2$ ,  $b = 5$ ,  $c = -4$  : équation  $2x + 5 = -4$

$$\blacktriangleright x = \frac{-4 - 5}{2} = \frac{-9}{2} = \boxed{-4,5}$$

b. La solution est  $x = -4,5$ . Vérification :  $2 \times (-4,5) + 5 = -9 + 5 = -4 \checkmark$

2. Pour  $a = 0$ ,  $b = 5$ ,  $c = 7$  : l'instruction `assert (a != 0)` lève une `AssertionError` et stoppe le programme. C'est un comportement **voulu** : si  $a = 0$ , l'équation  $0x + 5 = 7$  n'a pas de solution (en divisant par  $a = 0$  on obtiendrait une division par zéro).

### Exercice 7 — Les œufs

Tests du programme `print(n // 6)` :

| $n$ | $n // 6$ | Verdict   | Explication                                            |
|-----|----------|-----------|--------------------------------------------------------|
| 6   | 1        | Correct   | 6 œufs = 1 boîte pleine                                |
| 8   | 1        | Incorrect | 8 œufs nécessitent 2 boîtes (1 pleine + 1 avec 2 œufs) |
| 12  | 2        | Correct   | 2 boîtes pleines                                       |
| 13  | 2        | Incorrect | 13 œufs nécessitent 3 boîtes                           |
| 15  | 2        | Incorrect | 15 œufs nécessitent 3 boîtes                           |

1. Le programme est correct uniquement quand  $n$  est un **multiple de 6** (0, 6, 12, 18...). Dans tous les autres cas il **sous-estime** le nombre de boîtes.

2. Remplacer par `n // 6 + 1` n'est pas correct car cela **sur-estime** quand  $n$  est un multiple de 6 : pour  $n = 6$ , on obtiendrait  $1 + 1 = 2$  boîtes au lieu de 1.

3. Solution correcte n'utilisant que la division entière :

```
1 n = int(input("combien d'œufs : "))
2 print((n + 5) // 6)
```

*Justification* : ajouter 5 avant la division entière par 6 permet d'**arrondir au supérieur**.

► Si  $n$  est un multiple de 6,  $(n + 5) // 6$  donne le même résultat que  $n // 6$  : le reste de  $n + 5$  par 6 vaut 5, ce qui ne « franchit » pas le palier suivant.

► Si  $n$  n'est pas un multiple de 6, l'ajout de 5 fait passer au quotient supérieur, c'est-à-dire ajoute la boîte supplémentaire nécessaire.

Vérification :  $n = 6 \rightarrow 1$  ;  $n = 8 \rightarrow 2$  ;  $n = 12 \rightarrow 2$  ;  $n = 13 \rightarrow 3$  ;  $n = 15 \rightarrow 3$ .  $\checkmark$

*Remarque (pour plus tard)* : une fois les instructions conditionnelles étudiées, on pourra écrire la solution plus lisible `if n % 6 == 0 : ... else : ...`. À ce stade de l'année, la forme  $(n + 5) // 6$  a l'avantage de n'utiliser que la division entière, seule notion déjà construite.